Kalashree

AI, Data & Backend Systems Engineer

hello@kalashree.dev

github.com/kalashree225

India · open to remote

Building dependable AI services, resilient data pipelines, and clear product interfaces.

OPEN TO FULL-TIME ROLES AND FOCUSED CONTRACT WORK

SUMMARY

Software engineer working across intelligent backends, data workflows, and product surfaces. My approach is anchored in clarity: make AI models genuinely useful, make backend services observable and robust, and make interfaces remove friction rather than add it. Everything listed here links to a repository you can read.

AT A GLANCE

01 DOCUMENTED PROJECTS	04 WRITTEN ESSAYS
04 PRACTICE AREAS	18 STACKS & TOOLS LISTED

These are counts of things that exist in the public repositories — not uptime or latency claims, which would need production telemetry to be meaningful.

PRACTICE AREAS

AI & Backend Systems

Service boundaries, async data flow, and observability for long-running workloads.

Data & Insight Architecture

Ingestion, normalisation, semantic contracts, and evaluation gates.

Interfaces & Developer Tools

Interfaces that make complex backend behaviour legible to the people using it.

Systems Documentation

Writing the decision down: case studies, diagrams, and engineering notes.

SELECTED WORK

StockLedger

INVENTORY & ORDER SYSTEM

2026

An inventory and order system built around explicit domain logic and explicit transaction boundaries.

— Kept every stock mutation inside a single transaction with the inventory row locked for the decision, so concurrent orders cannot oversell.

— Managed schema changes with Alembic migrations written to be compatible across a rolling deploy.

— Moved receipts, notifications, and reporting to workers so the transaction lock stays short.

Python · PostgreSQL · Alembic · Workers

REPOSITORY ↗

SKILLS

LANGUAGES

Python · TypeScript · JavaScript · SQL · HTML5 / Modern CSS

AI & DATA

LLM Orchestration · FastAPI · ETL Pipelines · Pandas & NumPy · Vector Workflows · Model Evaluation

BACKEND & INFRASTRUCTURE

MongoDB · REST Architectures · Linux / Bash

FRONTEND & TOOLING

React 19 · Vite · React Router · Git Workflows

HOW I WORK

Useful over impressive

The best architecture is the simplest one that reliably solves the problem in front of you.

Explicit over implicit

Clear schemas, strict boundaries, and predictable APIs prevent the regressions that are hardest to find.

Respect the reader

Code and interfaces are read by people. Naming, layout, and documentation are part of the product.